

Unit 5

SMD Practical + Oral Exam Notes

UNIT 5 — SOFTWARE DESIGN PRINCIPLES AND PATTERNS

Topic 1: GRASP Principles (ALL 9 — CRITICAL FOR VIVA)

1. Simple Exam Definition:

GRASP (General Responsibility Assignment Software Patterns) is a set of 9 principles guiding **which class should be responsible for which operation** in an OO design.

2. Core Concept — All 9 GRASP Principles:

1. Information Expert

Rule: Assign a responsibility to the class that has the **information necessary** to fulfill it.

Example: Who should calculate a student's GPA?

- `Student` class has all the grades → `Student` is the Information Expert → `Student.calculateGPA()`.
- NOT the `Course` class (it doesn't know all grades of a student).

2. Creator

Rule: Class B should **create** an object of Class A if:

- B contains or aggregates A, OR
- B records A, OR
- B uses A closely, OR
- B has the initializing data for A.

Example: Who creates `EnrollmentRecord`?

- `EnrollmentController` records enrollments → it creates `EnrollmentRecord`.

3. Controller

Rule: The **first object** beyond the UI that receives and coordinates a system operation. It delegates work to other objects.

Types of Controllers:

- *Use Case Controller:* One controller per use case (e.g., `LoginController`, `EnrollController`).
- *Facade Controller:* One controller for the entire system or subsystem.
Example: Student clicks "Enroll" → `EnrollmentController.enroll()` is called, not the database directly.

4. Low Coupling

Rule: Minimize dependencies between classes. A class should not depend on too many other classes.

Why: If ClassA depends on ClassB, changing ClassB can break ClassA.

Good Example: `StudentService` only depends on `StudentRepository` (one dependency — low coupling).

Bad Example: `StudentUI` directly creates and calls `DatabaseConnection`, `StudentDAO`, `EmailService` (high coupling).

5. High Cohesion

Rule: Each class should have a **single, well-defined responsibility**. Don't put unrelated things in one class.

Good Example: `GradeCalculator` only calculates grades. `StudentDAO` only handles DB operations. (High Cohesion)

Bad Example: `StudentManager` calculates grades, sends emails, updates database, generates reports. (Low Cohesion — God class!)

6. Polymorphism

Rule: When behaviour varies by type, use **polymorphism** (method overriding) instead of if-else chains.

Example: Instead of `if (userType = "student") ... else if (userType = "professor") ...`

Use: `User.login()` overridden by `Student.login()` and `Professor.login()` — each handles it differently.

7. Pure Fabrication

Rule: When no real-world class is responsible for a task (to maintain low coupling and high cohesion), **invent** a helper class that doesn't exist in the domain.

Example: `DatabaseConnectionPool` — doesn't exist in the student domain, but we create it to manage DB connections without polluting domain classes.

Another example: `EmailNotificationService` — not a real-world concept, but a pure fabrication.

8. Indirection

Rule: Introduce an **intermediate object** to decouple two classes that would otherwise be tightly coupled.

Example: `EnrollmentController` sits between `StudentUI` and `StudentDAO`. The UI doesn't talk directly to the database.

Indirection and Pure Fabrication often work together.

9. Protected Variations

Rule: Identify points of **likely change** and wrap them behind a stable interface so changes don't propagate.

Example: Payment method changes often → create `PaymentStrategy` interface → any payment method (UPI, Card, Cash) implements it. Changing the payment method doesn't affect the rest of the system.

3. Summary Table — All 9 GRASP:

Principle	One-Line Rule	SMS Example
Info Expert	Who has the data, does the job	<code>Student.calculateGPA()</code>
Creator	Who records/contains A, creates A	<code>EnrollController</code> creates <code>EnrollRecord</code>
Controller	First object after UI coordinates flow	<code>EnrollmentController</code>
Low Coupling	Minimize dependencies	<code>StudentService</code> → only one DAO

Principle	One-Line Rule	SMS Example
High Cohesion	One class, one purpose	<code>GradeCalculator</code> only calculates
Polymorphism	Use override instead of if-else	<code>User.login()</code> per user type
Pure Fabrication	Invent a helper class	<code>DBConnectionPool</code>
Indirection	Add intermediate object	<code>Controller</code> between UI and DB
Protected Variations	Wrap change points with interface	<code>PaymentStrategy</code> interface

4. Examiner Viva Questions:

- What does GRASP stand for?
- Explain Information Expert with example.
- Difference between High Cohesion and Low Coupling (MOST POPULAR QUESTION).
- What is Pure Fabrication? Why do we need it?
- What is a Controller in GRASP?

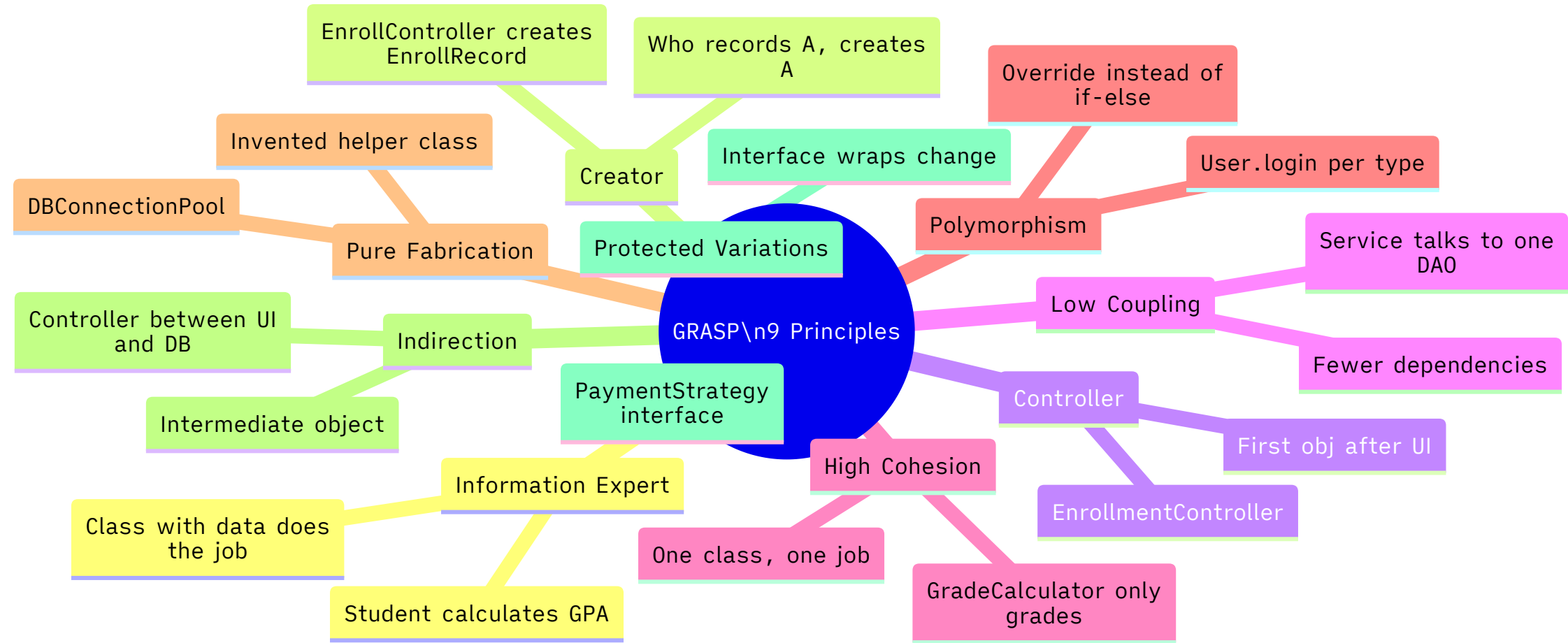
5. Strong Viva Answers:

- Answer:* High Cohesion means a class has **one focused responsibility**. Low Coupling means a class has **few dependencies** on other classes. We always want BOTH — High Cohesion AND Low Coupling.
- Answer:* Pure Fabrication is a GRASP principle where we **invent** a class that doesn't exist in the real world but is needed to keep the design clean. Example: `DBConnectionPool` is not a real academic concept, but we create it to handle database connections efficiently.

9. Difference Tables:

Feature	High Cohesion	Low Coupling
Focus	Inside a class (internal)	Between classes (external)
Goal	One purpose per class	Few dependencies between classes
Violation	"God Class" (does everything)	Classes tightly depending on each other
Analogy	Specialist doctor	Independent workers

 Visual: GRASP Principles Map



12. Oral Trap Questions:

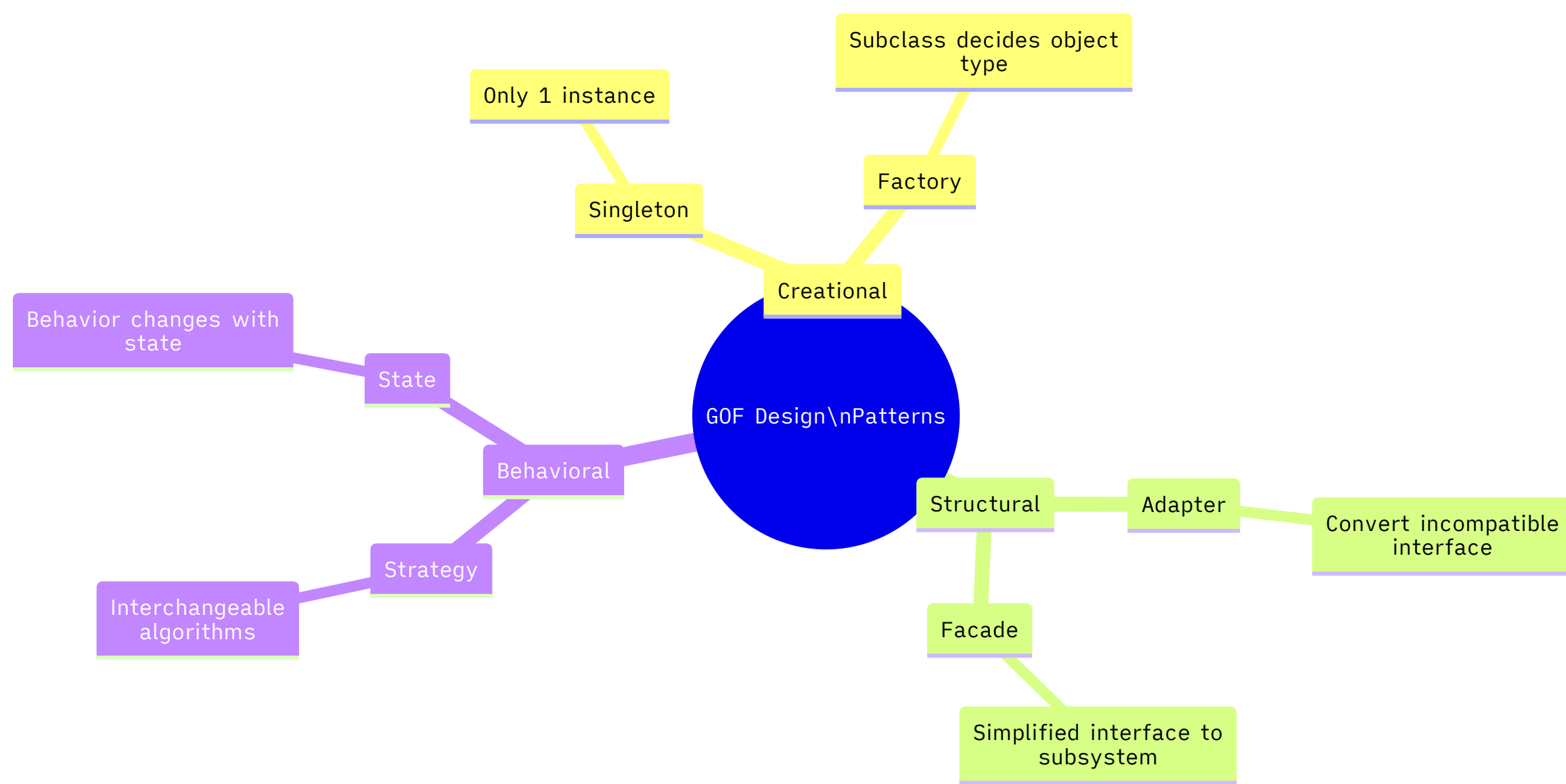
- *Examiner:* "Is 100% decoupling good?"
- *Answer:* "No, sir. Zero coupling means the classes cannot communicate at all. We aim for **low** coupling, not zero coupling."
- *Examiner:* "How does Protected Variations relate to design patterns?"
- *Answer:* "Many GOF patterns implement Protected Variations — for example, the Strategy pattern wraps the payment algorithm behind an interface so the algorithm can change without affecting the controller."

Topic 2: GOF Design Patterns (All 6 for Syllabus)

1. Simple Exam Definition:

Design Patterns (GOF — Gang of Four) are **reusable, proven solutions** to commonly recurring software design problems. They are categorized into Creational, Structural, and Behavioral.

 **Visual: GOF Pattern Categories**



Pattern 1: Singleton (Creational)

Concept: Ensures a class has **only ONE instance** and provides a global access point to it.

When to use: When exactly one object is needed to coordinate actions — e.g., a database connection, a logger, a configuration manager.

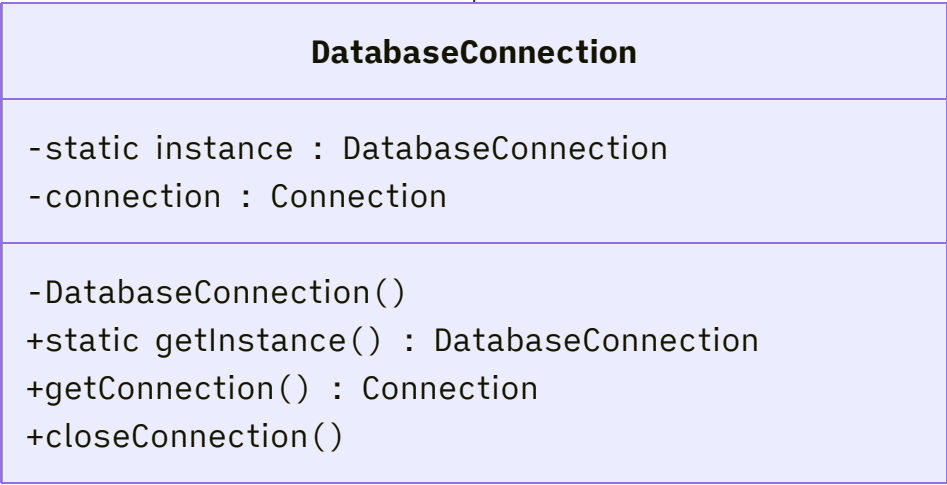
Structure:

- Private constructor (prevents `new ClassName()` from outside).
- Private static instance variable.
- Public static `getInstance()` method that returns the single instance.

Student Management System Example:

`DatabaseConnection` — we don't want 1000 students opening 1000 different DB connections. One connection is shared.

Only 1 instance exists.\nCall: DatabaseConnection.getInstance()\nConstructor is PRIVATE.



Examiner Viva Questions:

- What problem does Singleton solve?
- Why is the constructor private in Singleton?

Strong Viva Answers:

- Answer: Singleton ensures only one instance exists – the private constructor prevents external instantiation, and getInstance() always returns the same single object.

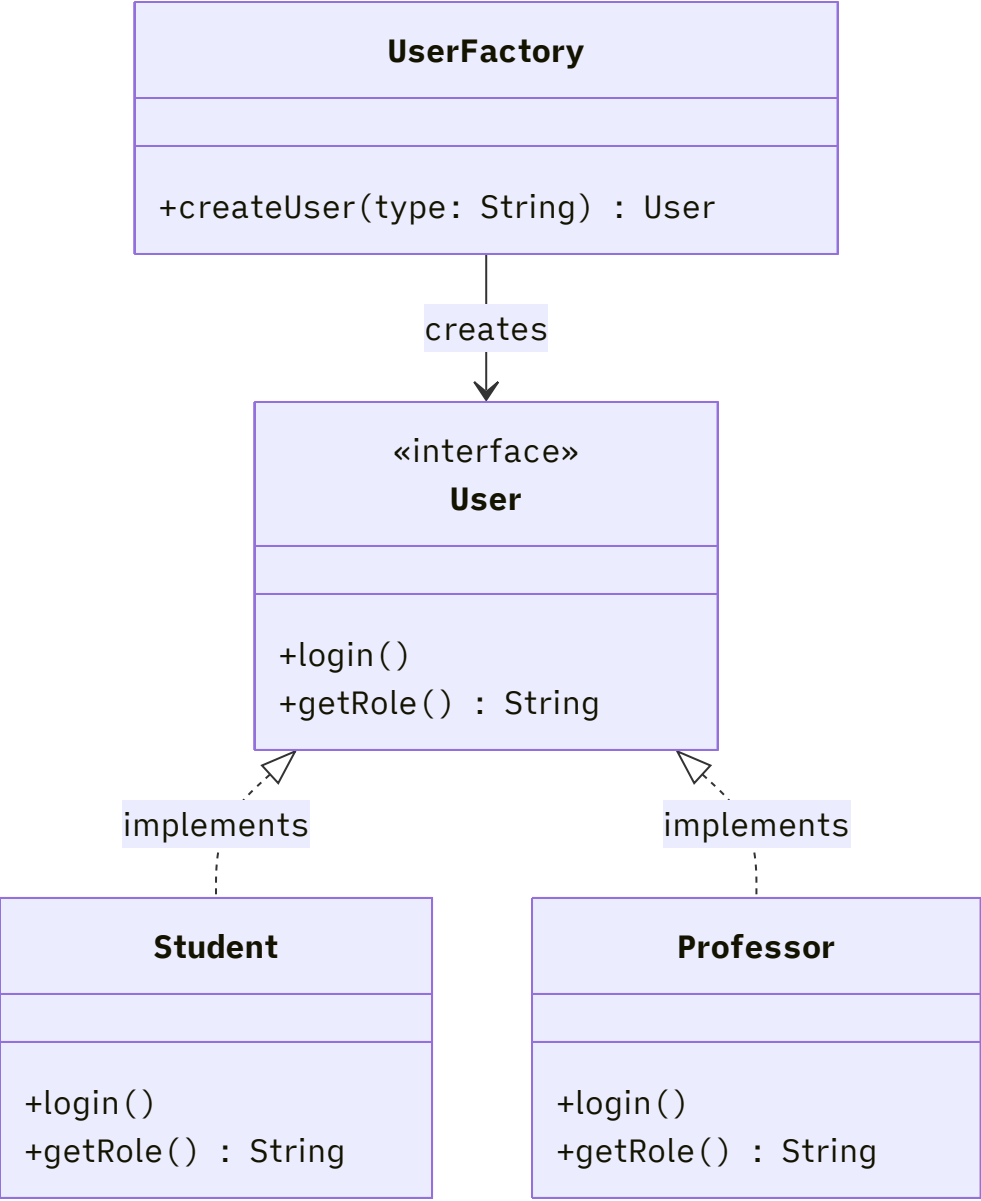
Pattern 2: Factory Method (Creational)

Concept: Defines an interface for creating objects, but lets **subclasses decide** which class to instantiate. The factory creates objects without specifying the exact class.

When to use: When the type of object to create is determined at runtime.

Student Management System Example:

UserFactory creates either a Student object or a Professor object based on the login type – the factory decides.



Oral Trap: "Difference between Singleton and Factory?"
Answer: "Singleton controls the number of instances (only one). Factory controls which **type** of object is created."

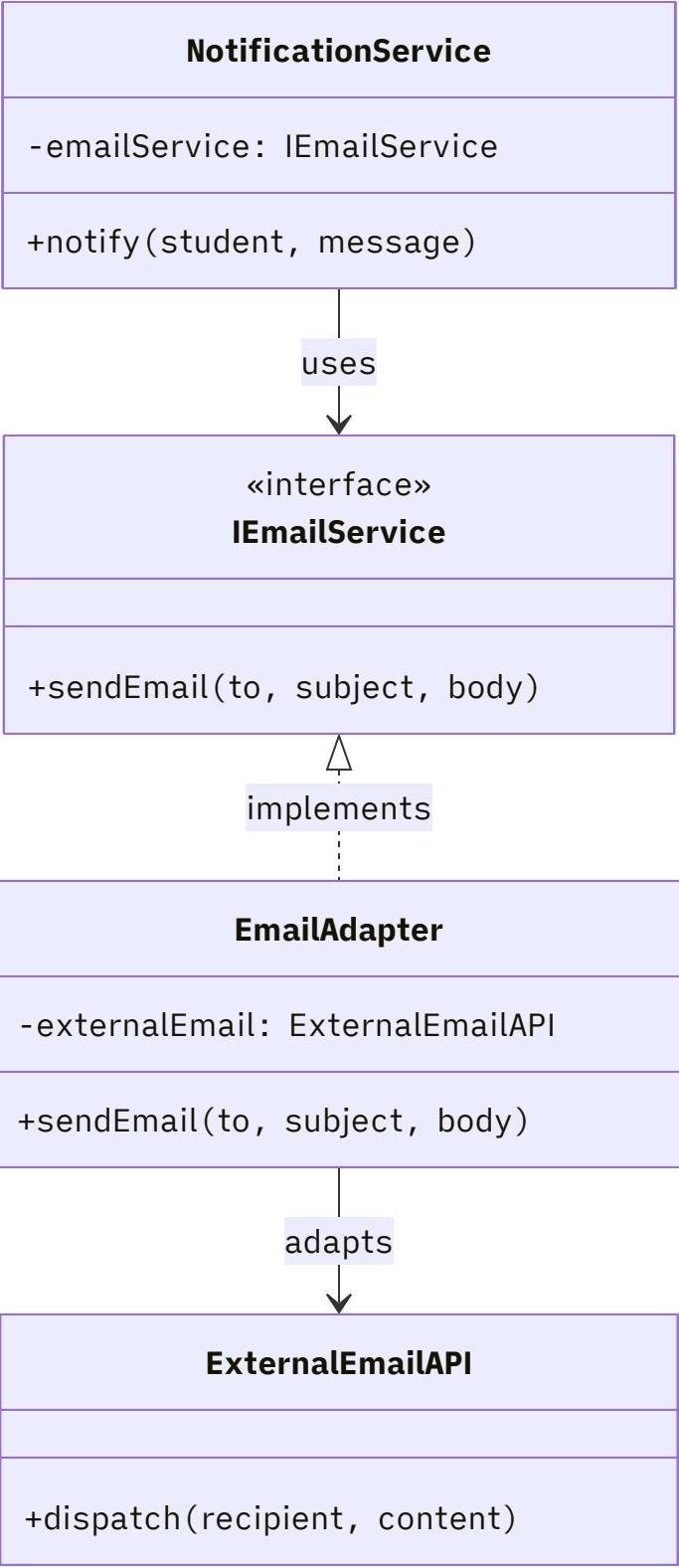
Pattern 3: Adapter (Structural)

Concept: Converts the interface of a class into another interface that the client expects. An adapter lets classes work together that couldn't otherwise due to **incompatible interfaces**.

Analogy: A 3-pin to 2-pin electrical adapter — the device and the socket are incompatible, but the adapter bridges them.

When to use: When integrating a third-party or legacy class whose interface doesn't match what your system expects.

Student Management System Example:
The Student System uses `sendEmail(to, subject, body)` but the external Email API only has `dispatch(recipient, content)`. We create an `EmailAdapter`.



Examiner Viva Questions:

- What problem does Adapter solve?
- Real-life analogy for Adapter?

Strong Viva Answer:

- *Answer:* Adapter converts an **incompatible interface** into a compatible one — like a translator between two people who speak different languages.

Pattern 4: Facade (Structural)

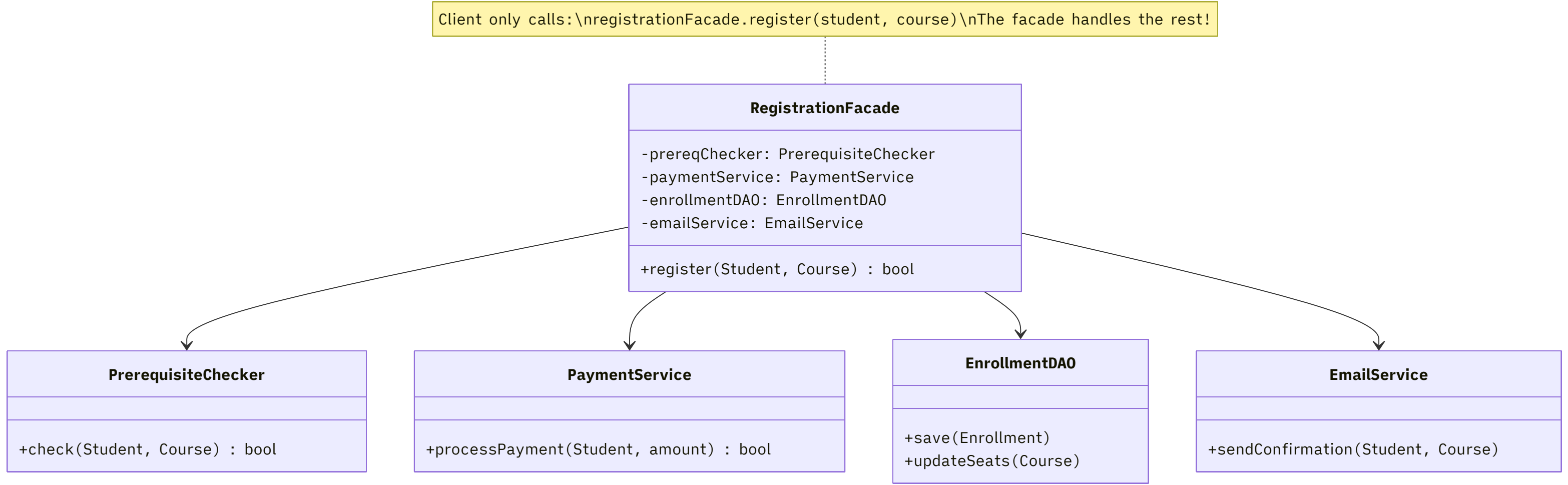
Concept: Provides a **simplified, unified interface** to a complex subsystem, hiding the complexity from the client.

Analogy: A TV remote — you press "Volume Up" without knowing the internal circuits.

When to use: When a subsystem has many complex classes and the client only needs simple, high-level operations.

Student Management System Example:

Enrollment involves: check prerequisites → verify payment → update seats → send email. Instead of calling all these manually, a `RegistrationFacade` provides one method `register(student, course)` that does all of this internally.



Oral Trap: "How is Facade different from Adapter?"

Answer: "Adapter changes an incompatible interface to a compatible one. Facade simplifies a complex interface — it doesn't change anything, it just hides complexity."

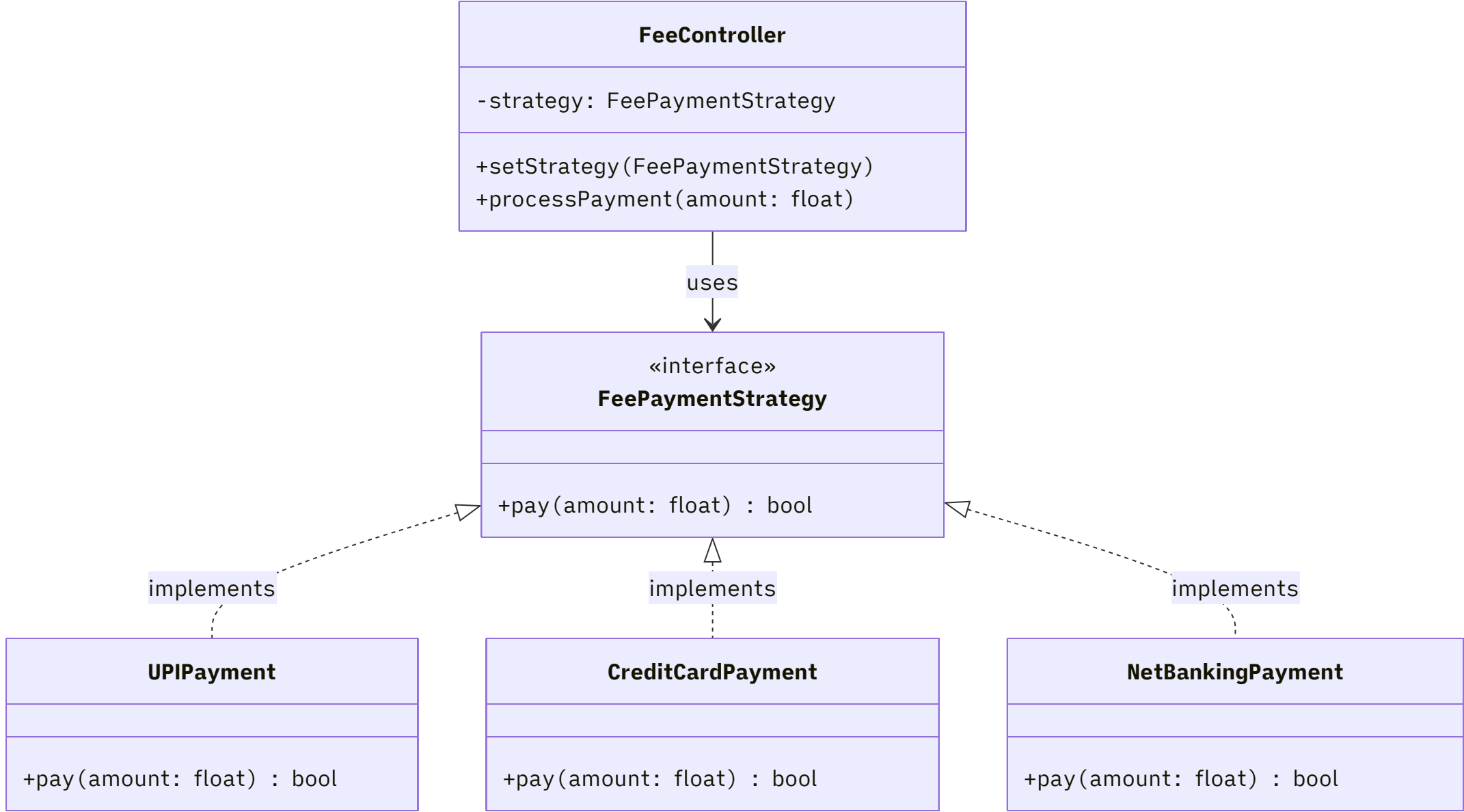
Pattern 5: Strategy (Behavioral)

Concept: Defines a **family of algorithms**, encapsulates each one, and makes them **interchangeable at runtime** without changing the client.

When to use: When you need to switch between different algorithms or behaviors at runtime.

Student Management System Example:

Fee payment can be done by UPI, Credit Card, or Net Banking. Instead of if-else, use `FeePaymentStrategy`.



Oral Trap: "Why not just use if-else for payment type?"
Answer: "If-else violates the Open/Closed Principle — every time we add a new payment method, we must modify the controller. With Strategy, we just add a new class implementing the interface. The controller doesn't change."

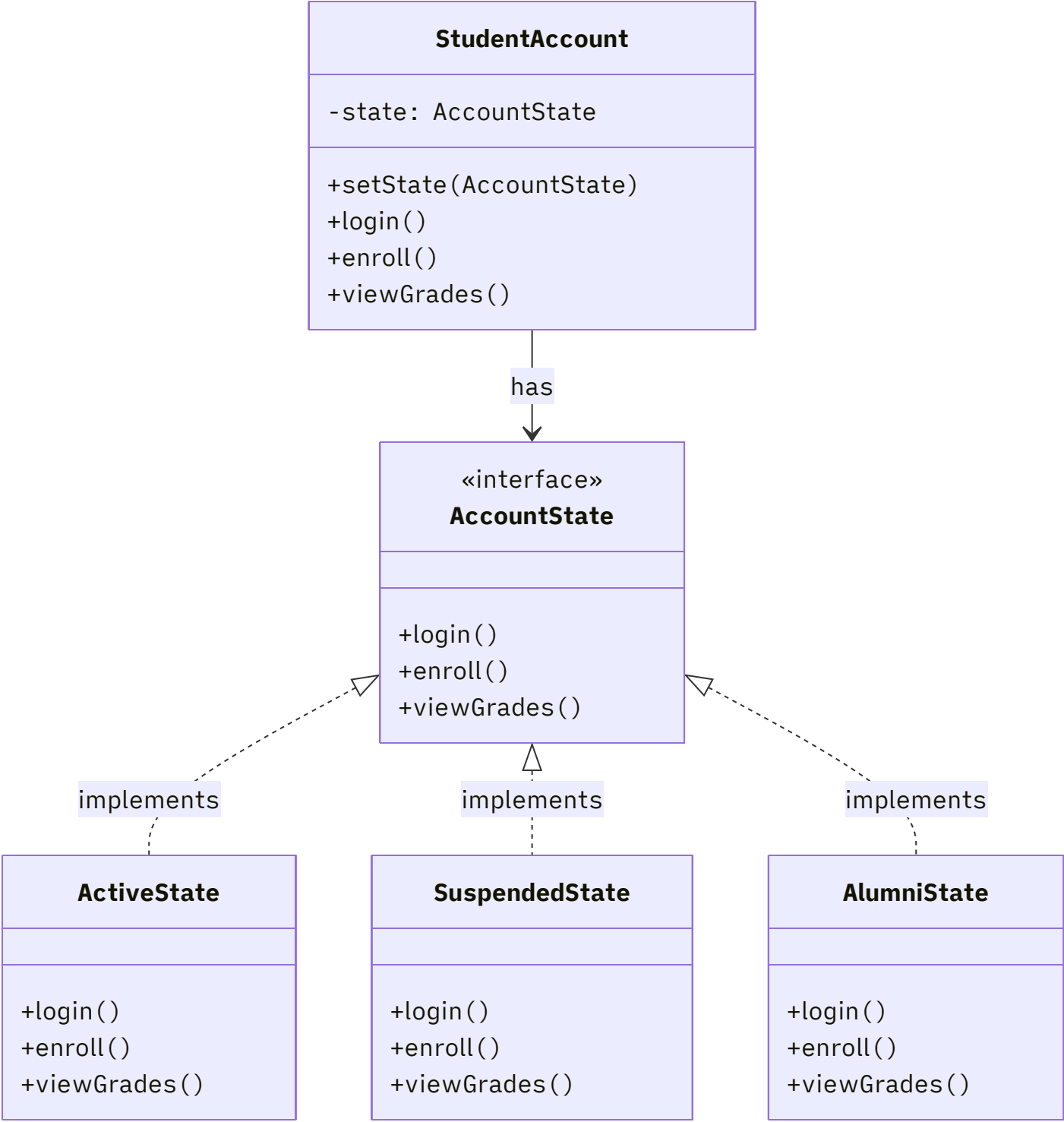
Pattern 6: State (Behavioral)

Concept: Allows an object to **change its behavior when its internal state changes**. The object appears to change its class.

When to use: When an object's behavior depends heavily on its state and changes at runtime.

Student Management System Example:

A `StudentAccount` behaves differently in `Active`, `Suspended`, and `Alumni` states — login, enrollment, and grade access work differently per state.



Oral Trap: "How is State pattern different from Strategy?"
Answer: "Both use a similar structure. Strategy lets the **client** choose and switch algorithms. State pattern switches behavior **automatically** based on the object's internal state — the client doesn't need to manage state transitions explicitly."

Final GOF Quick Reference:

Pattern	Type	Purpose	SMS Example
Singleton	Creational	Only one instance	DatabaseConnection
Factory	Creational	Subclass creates object	UserFactory → Student/Professor

Pattern	Type	Purpose	SMS Example
Adapter	Structural	Bridge incompatible interfaces	EmailAdapter for external API
Facade	Structural	Simplify complex subsystem	RegistrationFacade.register()
Strategy	Behavioral	Interchangeable algorithms	FeePaymentStrategy
State	Behavioral	Behavior per state	StudentAccount state behavior